

实验报告：Cache优化与指令级并行对程序性能的影响

邓耀飞 学号：2413404

摘要

本实验旨在探究现代计算机体系结构中Cache局部性和指令级并行对程序性能的影响。通过实现矩阵列内积的两种算法（逐列访问 vs 按行访问）和数组求和的两种算法（链式累加 vs 两路累加），在不同数据规模下进行性能测试。实验结果表明，Cache优化算法在矩阵规模为4096时获得5.68倍加速比，两路累加在小规模数组中获得约2倍加速比。本文还讨论了编译器优化对性能测量的影响，并提出了正确的测量方法。源代码已上传至GitHub仓库：<https://github.com/KiTa-Ynni/hpc-performance-lab>

目录

1	实验目的	2
2	实验平台	2
3	实验内容与算法设计	2
3.1	矩阵列内积	2
3.2	数组求和	2
4	实验方案	3
4.1	测试规模	3
4.2	计时方法	3
4.3	防止编译器优化	3
5	实验结果与分析	3
5.1	矩阵列内积性能对比	3
5.2	数组求和性能对比	4
6	进阶实验与讨论	4
6.1	编译器优化对性能测量的影响	4
6.2	性能瓶颈分析	4
7	实验结论	5
8	源代码	5

1 实验目的

1. 理解Cache局部性对矩阵运算性能的影响，对比“逐列访问”与“按行访问”两种算法的执行效率。
2. 理解指令级并行对数组求和性能的影响，对比“链式累加”与“两路累加”两种算法的执行效率。
3. 掌握在编译器优化（-O2）下进行性能测试的正确方法，避免“死代码删除”带来的测量误差。

2 实验平台

表 1: 实验平台配置

项目	配置
CPU	Intel Core i7-14650HX @ 2.20GHz
内存	16.0 GB
显卡	NVIDIA GeForce RTX 4060 Laptop GPU
操作系统	Windows 11 家庭中文版 (24H2)
编译器	g++ 8.1.0 (MinGW-W64)
优化级别	-O2
计时方式	std::chrono::high_resolution_clock, 自适应重复次数

3 实验内容与算法设计

3.1 矩阵列内积

问题描述：计算 $n \times n$ 矩阵 A 与向量 b 的每一列内积：

$$\text{result}[j] = \sum_{i=0}^{n-1} A[i][j] \times b[i]$$

算法设计：

- **平凡算法（逐列访问）：**外层循环遍历列，内层循环遍历行。内存访问模式为 $A[0][j], A[1][j], A[2][j] \dots$ ，空间局部性差。
- **Cache优化算法（按行访问）：**外层循环遍历行，内层循环遍历列，利用局部变量累加每列结果。内存访问模式为 $A[i][0], A[i][1], A[i][2] \dots$ ，空间局部性好，充分利用Cache line。

3.2 数组求和

问题描述：计算长度为 n 的数组所有元素之和。

算法设计：

- 平凡算法（链式累加）：单变量依次累加，每次迭代依赖上一次结果，无法利用指令级并行。
- 两路累加（指令级并行优化）：使用两个累加变量，分别处理偶数位和奇数位元素，最后合并。减少数据依赖，允许CPU同时执行两条加法指令。

4 实验方案

4.1 测试规模

表 2: 测试规模

测试项	规模范围
矩阵列内积	n = 256, 512, 1024, 2048, 4096
数组求和	n = 100000, 500000, 1000000, 5000000, 10000000

4.2 计时方法

采用自适应重复次数策略：

1. 先估算单次执行时间
2. 自动调整重复次数，使总测量时间稳定在约200-500ms
3. 取单次平均时间作为最终结果

4.3 防止编译器优化

在 -O2 优化级别下，未被使用的返回值会被编译器视为“无用代码”并删除。解决方案：将计算结果赋值给 `volatile` 变量，强制编译器保留计算过程。

5 实验结果与分析

5.1 矩阵列内积性能对比

表 3: 矩阵列内积性能对比

n	平凡算法 (ms)	Cache优化 (ms)	加速比
256	0.041	0.024	1.72
512	0.204	0.091	2.24
1024	1.030	0.359	2.87
2048	9.484	2.371	4.00
4096	58.169	10.242	5.68

分析：

1. 加速比随 n 增大而显著提升，从 1.72x 增长至 5.68x。
2. 当 $n = 4096$ 时，矩阵大小约为 $4096^2 \times 8 \text{ B} = 128 \text{ MB}$ ，远超 L3 Cache 容量（通常 8–32 MB）。平凡算法因频繁 Cache Miss 性能严重下降，而 Cache 优化算法仍保持较高效率。
3. 平凡算法按列访问时，相邻元素在内存中相隔一个列宽（约 32KB），破坏空间局部性；按行访问则连续读取，每次 Cache Line（64 字节）加载的 8 个元素全部被使用。

5.2 数组求和性能对比

表 4: 数组求和性能对比

n	平凡算法 (μs)	两路累加 (μs)	加速比
100000	43.85	22.59	1.94
500000	217.70	109.33	1.99
1000000	437.80	222.65	1.97
5000000	3050.78	2629.95	1.16
10000000	6622.43	4900.88	1.35

分析:

1. 在 $n \leq 10^6$ 时，两路累加稳定达到约 **2 倍加速比**。这是因为两路累加消除了数据依赖，允许 CPU 的两个浮点运算单元同时工作，达到指令级并行的理论极限。
2. 当 n 增大至 5×10^6 以上时，加速比下降至 1.16–1.35。此时内存访问时间成为主要瓶颈，计算优势被掩盖。
3. 验证输出显示两种算法结果一致（均为 n ），证明算法实现正确。

6 进阶实验与讨论

6.1 编译器优化对性能测量的影响

在 -O2 级别下，未使用的函数返回值会被编译器视为“无用代码”并删除。实验中，数组求和函数因仅返回 `double` 且未使用，被完全优化，测量结果为 0。

解决方法：使用 `volatile` 关键字强制保留计算结果：

```
volatile double result = Sum::naive(arr);
```

这体现了在高性能计算中，开发者必须理解编译器的优化行为，才能正确评估算法性能。

6.2 性能瓶颈分析

从表5可以看出，不同计算任务在不同规模下的性能瓶颈存在显著差异：

矩阵列内积（Cache Miss 主导）：当矩阵规模为 4096 时，数据量达到 128MB，远超 CPU 的 L3 Cache 容量（通常 8-32MB）。平凡算法因逐列访问导致频繁的 Cache Miss，每次访

表 5: 性能瓶颈分析

问题	瓶颈类型	优化效果
矩阵列内积	Cache Miss	5.68x 加速比
数组求和（小规模）	指令级并行	约 2x 加速比
数组求和（大规模）	内存带宽	加速比下降至 1.3x

问都可能触发从内存加载数据，耗时显著增加。而 Cache 优化算法通过按行访问，充分利用空间局部性，在 Cache 命中率上获得巨大优势，从而取得 5.68 倍加速比。

数组求和（小规模：指令级并行有效）：当数组规模不超过 100 万时，数据可以完全放入 Cache，内存访问时间不是瓶颈。此时，两路累加通过减少数据依赖，允许 CPU 的两个浮点运算单元同时工作，实现了接近 2 倍的理论加速比。这验证了现代超标量处理器的指令级并行能力。

数组求和（大规模：内存带宽瓶颈）：当数组规模达到 500 万以上时，数据总量超出 Cache 容量，每次求和都需要从内存读取数据。此时内存带宽成为主要限制因素，计算速度被数据供给速度拖慢。因此两路累加的计算优势被掩盖，加速比下降至 1.16-1.35 倍。

上述分析表明，性能优化策略需要根据问题规模和硬件特性灵活选择：计算密集型小规模问题适合指令级并行优化，而数据密集型大规模问题则需要优先考虑 Cache 优化和内存访问模式优化。

7 实验结论

- Cache 优化效果显著：**按行访问相比按列访问，在大规模矩阵运算中最高可获得 **5.68 倍** 加速比。性能提升来源于更好的空间局部性和更高的 Cache 命中率。
- 指令级并行有效：**两路累加在小规模数组中实现约 **2 倍** 加速比，验证了超标量处理器并行执行多条独立指令的能力。
- 性能瓶颈随规模变化：**小规模时计算能力是瓶颈，大规模时内存带宽成为主要限制因素。
- 正确测量需考虑编译器优化：**在高优化级别下，需通过 `volatile` 等手段防止被测代码被优化掉，确保测量结果真实有效。

8 源代码

本次实验的完整源代码已上传至 GitHub 仓库：

<https://github.com/K1Ta-Ynni/hpc-performance-lab>

仓库包含：

- `benchmark.cpp` —— 完整的性能测试代码
- `report.tex` —— 本实验报告源文件
- `README.md` —— 项目说明文档

参考文献

- [1] Hennessy, J. L., & Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.
- [1] 陈国良, 吴俊敏. (2018). 并行计算机体系结构. 高等教育出版社.
- [1] Intel Corporation. (2024). *Intel Core i7-14650HX Processor Specifications*.
- [1] GCC Team. (2023). *GCC Optimization Options*. <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>